Habit Tracking APP - Abstract

Object Oriented and Functional Programming with Python

Nathan Mersha Degineh, 102303079 , OOFPP (DLBDSOOFPP01)
Abstract, Phase 3
https://github.com/nathan-mersha/OOFPP

Overview

This project is a command line habit tracker application built with python for IU Object Oriented and functional Programming with Python course. The cli app helps the user to create, update, delete and analyze habits. It supports weekly and daily habit types. All habits and the user's name data is saved in a JSON file. There are 6 already defined habits, to help the evaluator to immediately analyze weekly and daily habits, without creating them. Having 6 predefined habits also helps the testing process.

Technical Approach

The cli application has 3 main modules : the tracker.py has 2 classes called HabbitJAL (as in DAL but for JSON) responsible for the actual write operation of the JSON file, HabbitController, responsible for create, delete and update actions of the habits and a helper.py that provides shared functions (like logging with color, parsing dates, enums for daily and weekly). The main.py contains the interactive menu and analytics functions.

The analytics module is built using python's functional programming paradigm (according to the assignment instructions).The project uses builtin python functions like filter, map and lambda function constructors. The streak function converts the completion dates to dates or weeks (based on the period type) and counts unbroken streaks.

Predefined Habits

Six habits are included (2 weekly and 4 daily) this helps the instructor to test the analytics, period filtering, streaks without creating any habits themselves.

Testing

A pytest unit tests and test screenshots `(/tests/test_screenshot.png)` is included in the `/tests` folder. These tests use their own json file, so they won't affect the actual application itself. The tests include, creating, marking completion, filtering, deletion, of the habits (and some custom tests for setting user name and retrieving) There are 10 tests and can be run with the command :

```
pytest tests/test.py -v
```

What went well

Because the code was separated between HabbitJAL (the json access layer) the HabitController and the main.py (presentation layer) it helped to easily test and keep the code clean. For example in the test function, I instantiated a new HabitJAL and HabitController with their own testing json file, and because the code was separated cleanly, reusing these classes became easier. In addition, having test data at the start of the project also helped greatly as testing streaks and analytics did not require creating new data every time.

Challenges

The main challenge I faced was the streak calculation logic, specially for weekly habits. Converting timestamps in to unique year weeks and checking if streak was broken or not was challenging at first, as the definition of week is not just 7 days gap, (if a user does his habit completion at midnight sunday, and does it again after 1 min in Monday morning, technically it should count as 2 week streak.) So to solve this, I first converted each timestamp into a unique week number, (making the first week in the year 2000 week 1, Note, if the user tries to import old data before the year 2000, the analytics feature won't work properly, but this is farfetched, i only mentioned it to acknowledge the gap) , remove duplicates, then count unbroken streaks. It took some time to reach this solution.

Another challenge was the file path handling, depending on where the test was invoked or where the application was run, you might get a file not found error, to solve this I used `os.path.dirname(__file__)` to get the absolute path of the file.

Another challenge was when i started working on this, my JSON file keeps getting corrupted on some unexpected places, (for example my cursor will be on the wrong place and it will add “}” on the wrong spot.) So to solve this (at least in the development process), I created a backup json file and when the system sees the file was corrupted it restores it using this file (of course in the real world this means loosing all your habbits, but i only did this for the development phase because there was a consistent file corruption) But now unless the user actually goes in to the json file and corrupt it himself, the application itself won't corrupt the file. Which without meaning to I have actually created a fail safe in the event that the user himself intentionally or unintentionally corrupts the json file himself.

Additional Features

The analytics menu provides 5 menus all in one place, the user can view a full habit list, filter by period, see longest streak for all habits, and a single habit with the best streak by daily or weekly.

The automatic json restore mechanism (although not required by the task) seems like a good failsafe in the event where the user intentionally or unintentionally corrupt the JSON file.

The message outputs are also color coded to give a more rich experience for the user. Warnings are in yellow, errors in red, successes in green and menus in blue.

The user can also customize his user name, which the bot will use to communicate with him, making the experience more personalized.